

Notebooks y asistentes de IA: guía práctica

Cómo trabajar con notebooks (.ipynb) en VS Code, Cursor y Google Colab, y cómo usar asistentes de IA (Claude, Codex, Gemini) para aprender más rápido – no para aprender menos.

CONTENIDO · 1. Qué es un notebook · 2. VS Code / Cursor · 3. Google Colab · 4. ¿Local o Colab? · 5. Asistentes de IA · 6. GitHub · 7. Casos de uso

1. Qué es un notebook y por qué lo usamos

Un archivo `.ipynb` (Jupyter Notebook) es un documento interactivo que mezcla **código, resultados y texto** en un mismo lugar. Es el formato estándar del trabajo analítico: en esta materia, todo lo que hagas va a vivir en notebooks.

La diferencia con un script `.py`: *un script es para cuando ya sabés qué querés hacer; un notebook es para cuando todavía no lo sabés*. Analizar datos es un loop – mirás, probás, ves el resultado, ajustás – y el notebook está diseñado para ese loop.

Estado vivo

Cargás el dataset una vez y seguís trabajando sobre él. No re-ejecutás todo desde cero con cada cambio.

Resultados inline

Tablas y gráficos aparecen debajo de cada celda, al lado del código que los generó.

Narrativa

Celdas de texto (Markdown) entre el código: el notebook es a la vez el análisis y el informe.

2. Trabajar en local: VS Code o Cursor

Cursor es un fork de VS Code: **todo lo de esta sección aplica idéntico a los dos**.

Preparar el entorno (una sola vez)

1. Instalá **Python** en tu máquina si no lo tenés (python.org o Anaconda).
2. En VS Code, abrí la vista de extensiones (`⌘` `⇧` `X` / `Ctrl` `⇧` `X`) e instalá **Python** y **Jupyter** (las dos de Microsoft).
3. Instalá las librerías de la materia desde la terminal integrada (`Ctrl` ```):

```
$ pip install pandas numpy matplotlib seaborn
```

Crear un notebook y correrlo

1. **Crear el archivo:** `⌘` `⇧` `P` → escribí “Create: New Jupyter Notebook”. O simplemente creá un archivo nuevo con extensión `.ipynb` (ej.: `tp1-exploracion.ipynb`).
2. **Elegir el kernel:** click en *Select Kernel* (arriba a la derecha) y elegí tu instalación de Python. El kernel es el Python que corre atrás del notebook: el notebook es la cara, el kernel es el cerebro.
3. **Ejecutar celdas:** escribí código y apretá `⇧` `Enter`. El resultado aparece debajo. Con la barra superior podés agregar celdas de código o de Markdown.

```

# TP1 – Exploración de ventas

[1] import pandas as pd
    df = pd.read_csv("ventas.csv")
    df.head()

[2] df["monto"].describe() # la variable df sigue viva de la celda anterior
    count    1.500e+03
    mean     2.847e+02 ...
  
```

La vista de notebook en VS Code / Cursor: barra de celdas arriba, kernel a la derecha, y el número [1], [2]... indica el orden en que se ejecutó cada celda.

ATAJO	QUÉ HACE
Enter	Ejecuta la celda y pasa a la siguiente
Enter / Ctrl Enter	Ejecuta la celda sin moverse
Esc luego A / B	Nueva celda arriba / abajo
Esc luego M / Y	Convierte la celda a Markdown / a código
Esc luego D D	Borra la celda

LA TRAMPA DEL ESTADO OCULTO

Las variables viven en el **kernel**, no en las celdas. Si definís `x = 10`, ejecutás y después **borrás la celda**, **x sigue existiendo**. Y si ejecutás celdas en desorden, el notebook puede dar resultados que nadie puede reproducir – incluido tu yo de mañana. Los números [1] [2] [7] a la izquierda muestran el orden real de ejecución.

REGLA DE LA MATERIA

Antes de entregar cualquier notebook: **Restart + Run All** (reiniciar el kernel y correr todo de arriba a abajo). Si no corre limpio de punta a punta, no está terminado.

3. Trabajar en la nube: Google Colab

Colab es Jupyter corriendo en servidores de Google, gratis y desde el navegador. No instalás nada: entrás con tu cuenta de Google y ya estás ejecutando Python.

- 1 Entrá a **colab.research.google.com** y logueate con tu cuenta de Google.
- 2 Archivo → *Nuevo notebook*. Se guarda automáticamente en tu Google Drive.
- 3 Ejecutá la primera celda con Enter o el botón ▶. La primera vez tarda unos segundos: Colab está *conectando* una máquina virtual para vos (fijate el estado arriba a la derecha).
- 4 **Subir tus datos:** abrí el panel *Archivos* (barra izquierda) y arrastrá tu CSV. Ojo: esos archivos se borran cuando la sesión termina. Para datasets que usás siempre, montá tu Drive con el botón del panel o con `from google.colab import drive; drive.mount('/content/drive')`.

tp1_exploracion.ipynb

Conectar 3

```
import pandas as pd
df = pd.read_csv("ventas.csv")
df.head()
```

◆ Gemini – “Genera código con IA”

Un notebook en Colab: botón de ejecutar en cada celda, estado de conexión arriba a la derecha, y Gemini integrado en la interfaz.

LAS SESIONES SE REINICIAN

Si dejás Colab inactivo un rato, la máquina virtual se apaga: **perdés las variables y los archivos subidos** (el notebook no – queda en Drive). Al volver: reconectar y *Entorno de ejecución* → *Ejecutar todo*. Es la misma lógica de Restart + Run All.

4. ¿Local o Colab?

	LOCAL (VS CODE / CURSOR)	GOOGLE COLAB
Setup inicial	Instalar Python + extensiones (~15 min, una vez)	Cero – navegador y cuenta de Google
Tus archivos	En tu disco, siempre disponibles	Hay que subirlos o montar Drive; se borran al cerrar sesión
Internet	Funciona offline	Imprescindible
Potencia	Tu compu generalmente es más potente para el trabajo del día a día	El runtime gratuito es modesto; podés pedir GPU/TPU para ML pesado
Librerías	Instalás y controlás las versiones	Muchas preinstaladas; lo que falte, <code>!pip install</code> en cada sesión
Colaborar / compartir	Git o enviar el archivo	Un link, como Google Docs
Asistente de IA	El que quieras: Claude, Codex, Copilot...	Gemini (integrado)

RECOMENDACIÓN PRÁCTICA

En la materia **proponemos trabajar en local**: es como se suele trabajar profesionalmente (control total, tus archivos, tu asistente de IA). **Colab** queda para cuando necesites GPU, quieras compartir un análisis con un link, o estés en una máquina que no es la tuya.

5. Asistentes de IA: qué opciones tenés

Trabajando en local podés elegir asistente; en Colab viene Gemini integrado. Todos hacen lo mismo en esencia: ven tu código y tus errores, y conversás con ellos sin salir del editor.

ASISTENTE	DÓNDE CORRE	CÓMO EMPEZAR
Claude (Claude Code)	Extensión de VS Code / Cursor, o terminal	Instalá la extensión “Claude Code” desde el marketplace y logueate con tu cuenta de Claude. En terminal: <code>npm install -g @anthropic-ai/claude-code</code> y después <code>claude</code> .

ASISTENTE	DÓNDE CORRE	CÓMO EMPEZAR
Codex (OpenAI)	Extensión de VS Code / Cursor, o terminal	Extensión “Codex - OpenAI” desde el marketplace, logueate con tu cuenta de ChatGPT. En terminal: <code>npm install -g @openai/codex</code> y después <code>codex</code> .
Gemini	Integrado en Colab	Botón  en la interfaz de Colab, o “Generar” en una celda vacía. No requiere instalación.

¿**Extensión o terminal?** La extensión abre un panel de chat dentro del editor y ve el archivo que tenés abierto: ideal para empezar. La terminal (CLI) es el mismo asistente con más autonomía – puede leer todo el proyecto, crear archivos y correr comandos. Empezá por la extensión; pasate a la CLI cuando te quede cómoda.

Cómo usarlos bien (esto es lo importante)

Un asistente de IA es **el mejor tutor que tuviste**: responde al instante, no se cansa, y puede explicarte lo mismo de cinco formas distintas. Y es también **la peor trampa** si lo usás de fotocopidora: pegar el enunciado y entregar lo que devuelve es entrenar al modelo, no entrenarte vos. En el parcial y en la entrevista de trabajo no está el asistente.

TAMBIÉN PARA INSTALAR Y CONFIGURAR

Si algo del setup falla – una librería que no instala, Python que no aparece, un error raro de entorno – no pierdas una hora googleando: **de última, pegale el error a Claude** y pedile que lo resuelva con vos paso a paso.

6. Compartir tu trabajo: GitHub

En la práctica, los proyectos de datos viven en repositorios de **Git** y se comparten por **GitHub**: historial de versiones, trabajo en equipo y vidriera de tu trabajo (tu perfil de GitHub es parte de tu CV). La buena noticia: **no hace falta memorizar comandos de git** – es exactamente el tipo de tarea que conviene hacer con Claude.

- 1 Creá tu cuenta en **github.com** (una sola vez).
- 2 **Subir el proyecto**: abrí Claude parado en la carpeta de tu proyecto y pedíselo directo:

PROMPT Creá un repositorio en GitHub llamado `tp1-analitica` y subí este proyecto.

Claude corre los comandos de git por vos (`init`, `commit`, `push`) y te muestra cada paso. Leé lo que va haciendo: es la forma más fácil de aprender git de paso.

- 3 **Guardar avances**: cada vez que termines algo que funciona:

PROMPT Guardá mis cambios con un mensaje descriptivo y subilos a GitHub.

- 4 **Branches – probar sin romper**: una *branch* es una línea de trabajo paralela: experimentás sin tocar lo que ya funciona en `main`, y si sale bien se une (*merge*) de vuelta.

PROMPT Creá una branch llamada `prueba-graficos` para probar otra visualización.

PROMPT Funcionó – mergeá esta branch a `main` y volvé a `main`.

- 5 **Compartir**: pasás el link del repo. GitHub renderiza los `.ipynb` directamente en el navegador – quien lo abre ve tu análisis con tablas y gráficos, sin instalar nada.

ANTES DE CADA SUBIDA

Restart + Run All para que lo que subís sea reproducible. Y ojo con los datos: los datasets pesados o con información sensible no van al repo – pedile a Claude que arme un `.gitignore` para excluirlos.

7. Casos de uso concretos (con el prompt)

ENTENDER UN ERROR

Pedí la explicación antes que la solución – ahí está el aprendizaje.

Me tira este `KeyError`. Explicame por qué pasa y no me des la solución todavía.

ENTENDER CÓDIGO AJENO

Para el código de la cátedra, de un compañero o de Stack Overflow.

Explicame esta celda línea por línea, como si fuera mi primer día con pandas.

EXPLORAR UN DATASET

Que te arme el esqueleto del análisis – el criterio lo ponés vos.

Cargué `ventas.csv` en `df`. Proponeme 5 preguntas interesantes para explorar este dataset y el código de la primera.

MEJORAR UN GRÁFICO

De “gráfico que salió” a “gráfico que comunica”.

Este gráfico muestra ventas por mes. ¿Cómo lo hago más legible? Título, ejes, formato de números.

COMPARAR ALTERNATIVAS

Para construir criterio, no solo resolver.

Dame 3 formas distintas de hacer este `groupby` en pandas y los trade-offs de cada una.

ESTUDIAR PARA EL PARCIAL

Invertí los roles: que el asistente pregunte.

Haceme 5 preguntas sobre este notebook para verificar que entendí lo que hice. Corregime las respuestas.

Checklist antes de entregar un notebook

- ☐ **Restart + Run All** – corre limpio de arriba a abajo, sin errores.
- ☐ Tiene celdas de **Markdown** que cuentan qué hiciste y qué encontraste.
- ☐ El archivo tiene un **nombre con sentido** (`tp1-exploracion.ipynb` , no `Untitled-3.ipynb`).
- ☐ Podés **explicar cada celda** sin mirar el chat de la IA.